



Universidad
Tecnológica
del Perú

PRIMERA UNIDAD DE APRENDIZAJE 3: Patrones estructurales

CURSO: DISEÑO DE PATRONES

SESIÓN N° 09:

**Patrones Estructurales:
Patrón Decorator. Patrón Composite**

DOCENTES DE UTP
SISTEMAS



Lo que debemos tener en cuenta



5 minutos



¿QUÉ SON PATRONES decorator y composite?

Decorator: Permite agregar funcionalidades adicionales a un objeto de manera dinámica sin modificar su estructura. Es útil para extender las capacidades de un objeto de forma flexible.

Composite: Permite tratar de manera uniforme objetos individuales y composiciones de objetos. Es útil para representar jerarquías de objetos, donde tanto los objetos simples como las composiciones pueden ser tratados de la misma manera.



¿Qué vimos en la sesión anterior?



5. Relaciona cada patrón de diseño con su descripción correspondiente.

Move the answers to the correct boxes

Permite que dos interfaces incompatibles trabajen juntas convirtiendo la interfaz de una clase en otra que el cliente espera.

Proporciona una interfaz simplificada para un conjunto de interfaces en un subsistema, facilitando su uso.

Se utiliza cuando se necesita una interfaz simple para un subsistema complejo o para reducir el acoplamiento entre los clientes y las clases del subsistema.

Se emplea cuando se necesita utilizar una clase existente pero su interfaz no es compatible con el sistema actual.

Uso del Adapter

Drag and drop
an answer here

Patrón Adapter

Drag and drop
an answer here

Patrón Facade

Drag and drop
an answer here

Uso del Facade

Drag and drop
an answer here



Unidad de aprendizaje 3: Patrones estructurales.	Semana 8,9,10 y 11
Logro específico de aprendizaje: Al finalizar la unidad el participante aplica los patrones estructurales para la solución de problemas.	
Temario: • Patrones Estructurales: Introducción a patrones estructurales. Patrón Adapter. Patrón Facade. • Patrones Estructurales: Patrón Decorator. Patrón Composite. • Consolidación de temas (Repaso). • Patrones Estructurales: Patrón Proxy. Patrón Bridge.	

Al final de la sesión el estudiante aplica patrones estructurales:

Patrón Decorator. Patrón Composite en un contexto definido



Decorator: Es importante porque permite añadir o modificar el comportamiento de un objeto en tiempo de ejecución sin cambiar su código original. Esto proporciona flexibilidad para extender funcionalidades de manera modular y reutilizable.

Composite: Es importante porque facilita el manejo y la manipulación de estructuras jerárquicas de objetos (como árboles) de manera uniforme. Permite tratar objetos individuales y grupos de objetos de manera coherente, simplificando la gestión de estructuras complejas.



¡Que no se te escape nada de tu clase!:

SESIÓN N° 09:

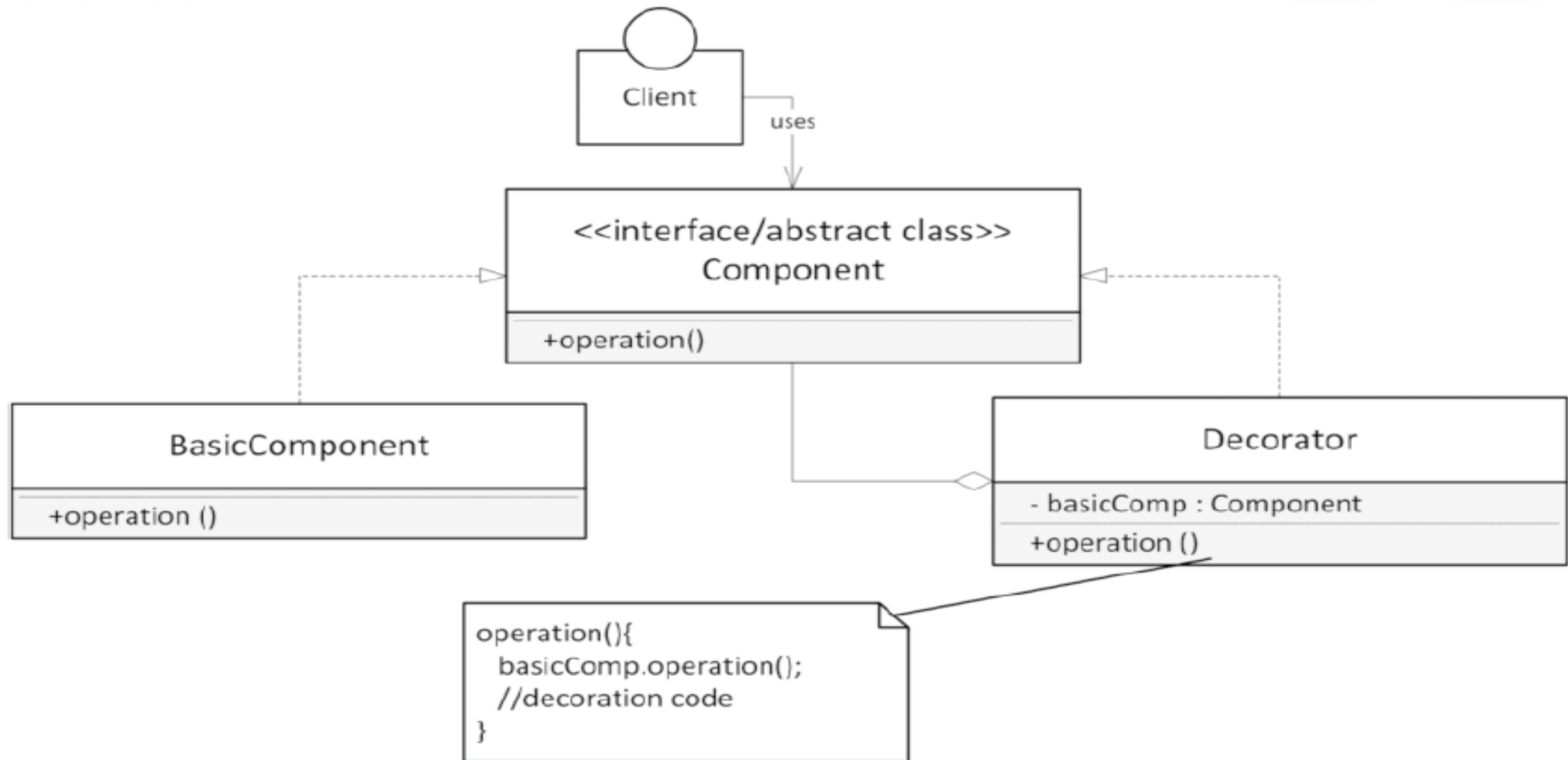
**Patrones Estructurales:
Patrón Decorator. Patrón Composite**

Este patrón proporciona una forma de manejar tareas de manera jerárquica comenzando con operaciones más rudimentarias.

Es un diagrama de clases que describe todos los elementos relevantes y sus relaciones. BasicComponent y Decorator son ambos tipos derivados de Component, y el último agrega una instancia del primero. La operación del método en Decorator está “decorada” con la de BasicComponent.

En otras palabras, un decorador es una forma de basar una operación en operaciones de otro componente.

Decorator es estructuralmente como Proxy, pero los dos patrones se basan en intenciones de diseño muy diferentes. Para Decorator, el interés del cliente está en el objeto que se agrega, mientras que para Proxy, el objetivo del cliente es el objeto agregado.



Caso de Estudio: Transporte

Supongamos que tenemos una interfaz Transport y queremos agregar características adicionales como "Seguro" y "Rastreo" a los transportes.

```
// Interfaz base
public interface Transport {
    String getDescription();
    double cost();
}

// Implementación concreta
public class BasicTransport implements Transport {
    public String getDescription() {
        return "Basic Transport";
    }
    public double cost() {
        return 100.0;
    }
}

// Decorador abstracto
public abstract class TransportDecorator implements Transport {
    protected Transport decoratedTransport;
    public TransportDecorator(Transport decoratedTransport) {
        this.decoratedTransport = decoratedTransport;
    }
    public String getDescription() {
        return decoratedTransport.getDescription();
    }
    public double cost() {
        return decoratedTransport.cost();
    }
}
```

Caso de Estudio: Transporte

Supongamos que tenemos una interfaz Transport y queremos agregar características adicionales como "Seguro" y "Rastreo" a los transportes.

// Decorador concreto para seguro

```
public class InsuranceDecorator extends TransportDecorator {  
    public InsuranceDecorator(Transport decoratedTransport) {  
        super(decoratedTransport);  
    }  
    public String getDescription() {  
        return decoratedTransport.getDescription() + ", with Insurance";  
    }  
    public double cost() {  
        return decoratedTransport.cost() + 20.0;  
    }  
}
```

// Decorador concreto para rastreo

```
public class TrackingDecorator extends TransportDecorator {  
    public TrackingDecorator(Transport decoratedTransport) {  
        super(decoratedTransport);  
    }  
    public String getDescription() {  
        return decoratedTransport.getDescription() + ", with Tracking";  
    }  
    public double cost() {  
        return decoratedTransport.cost() + 15.0;  
    }  
}
```

// Clase cliente

```
public class Client {  
    public static void main(String[] args) {  
        Transport transport = new BasicTransport();  
        System.out.println(transport.getDescription() + " costs " +  
            transport.cost());  
  
        Transport insuredTransport = new InsuranceDecorator(new  
            BasicTransport());  
        System.out.println(insuredTransport.getDescription() + " costs " +  
            insuredTransport.cost());  
  
        Transport trackedAndInsuredTransport = new TrackingDecorator(new  
            InsuranceDecorator(new BasicTransport()));  
        System.out.println(trackedAndInsuredTransport.getDescription() + "  
costs " + trackedAndInsuredTransport.cost());  
    }  
}
```

Caso de Estudio: Transporte

Supongamos que tenemos una interfaz Transport y queremos agregar características adicionales como "Seguro" y "Rastreo" a los transportes.

```
// Clase cliente
public class Client {
    public static void main(String[] args) {
        Transport transport = new BasicTransport();
        System.out.println(transport.getDescription() + " costs " + transport.cost());

        Transport insuredTransport = new InsuranceDecorator(new BasicTransport());
        System.out.println(insuredTransport.getDescription() + " costs " + insuredTransport.cost());

        Transport trackedAndInsuredTransport = new TrackingDecorator(new InsuranceDecorator(new BasicTransport()));
        System.out.println(trackedAndInsuredTransport.getDescription() + " costs " + trackedAndInsuredTransport.cost());
    }
}
```

BasicTransport: Implementa la interfaz Transport con una descripción y un costo base.

TransportDecorator: Clase abstracta para los decoradores, que añade funcionalidades a un transporte existente.

InsuranceDecorator y TrackingDecorator: Decoradores concretos que añaden seguro y rastreo, respectivamente.

Client: Muestra cómo se pueden combinar decoradores para enriquecer las funcionalidades de un transporte

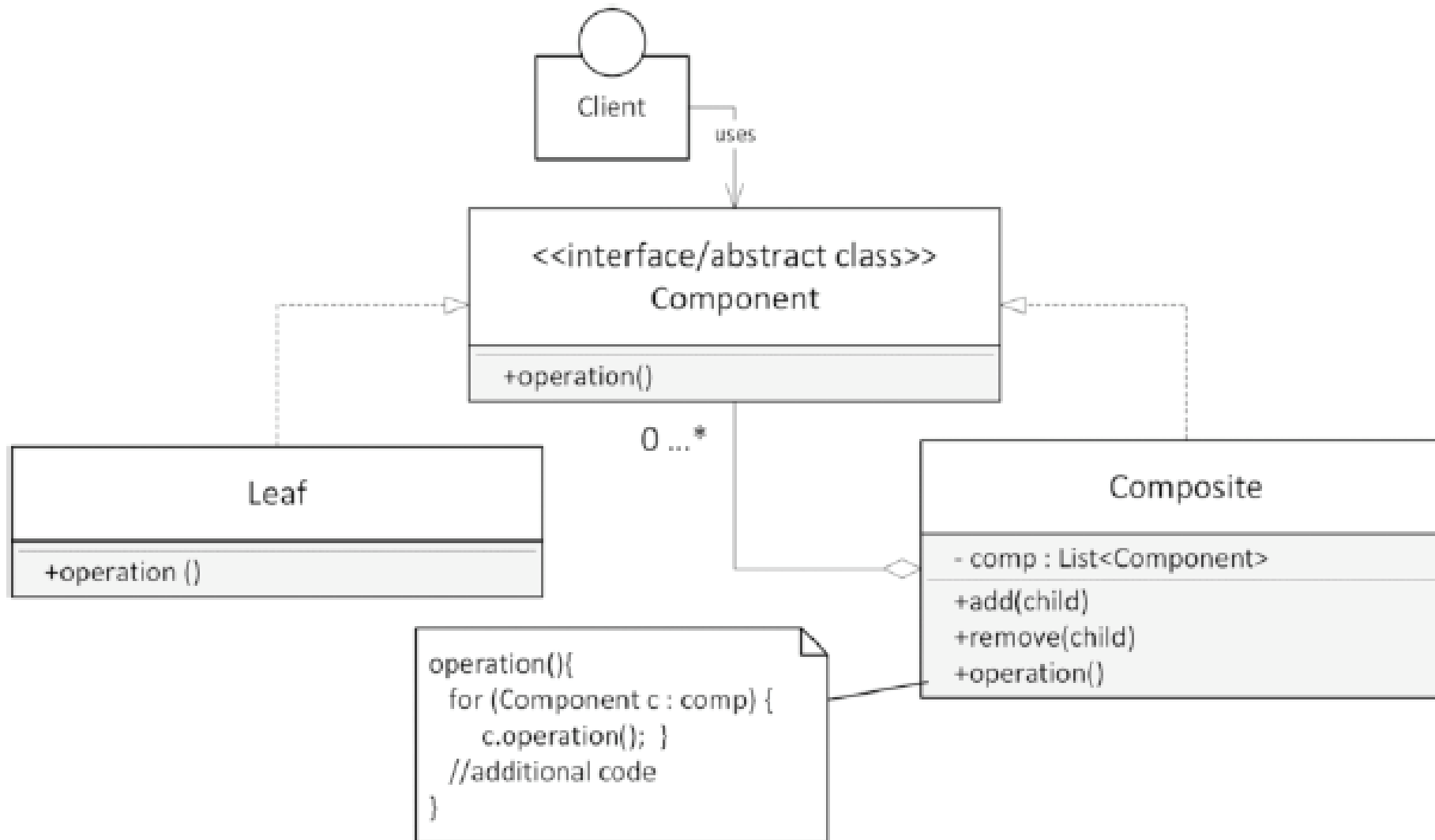
Ciertos elementos de software son jerárquicos, como la simulación de un motor, una tarea de un curso o la estructura de gestión de una organización. Dichos elementos se crean en función de relaciones parte-todo.

Por ejemplo, una tarea de un curso puede constar de varias partes; cada una puede tener su propia división de partes (cada parte es una tarea de pequeña escala).

Por lo tanto, resulta útil utilizar el mismo tipo de objeto para todos los elementos de la misma composición, de modo que el código del cliente pueda tratarlos de manera coherente para ejecutar operaciones que se implementan de forma polimórfica.

Aunque un decorador también tiene una relación parte-todo, ya que un componente básico es parte de un todo decorado, solo ofrece una composición "vertical" simple entre el todo y sus partes.

La figura ilustra el patrón Composite, una generalización del patrón Decorator con una colección agregada de instancias de Component.



Caso de Estudio: Sistema de Gestión de Pacientes

Supongamos que tenemos un sistema para gestionar pacientes y sus tratamientos, donde tanto los tratamientos individuales como los conjuntos de tratamientos (compositores) deben ser tratados de manera uniforme.

Patrones estructurales – Composite - ejemplo

```
// Componente base
public interface Treatment {
    String getDescription();
    double cost();
}

// Tratamiento individual
public class SingleTreatment implements Treatment {
    private String description;
    private double cost;

    public SingleTreatment(String description, double cost) {
        this.description = description;
        this.cost = cost;
    }

    public String getDescription() {
        return description;
    }

    public double cost() {
        return cost;
    }
}
```

Patrones estructurales – Composite - ejemplo

```
// Tratamiento compuesto (para varios tratamientos)
import java.util.ArrayList;
import java.util.List;
public class CompositeTreatment implements Treatment {
    private List<Treatment> treatments = new ArrayList<>();
    public void addTreatment(Treatment treatment) {
        treatments.add(treatment);
    }
    public void removeTreatment(Treatment treatment) {
        treatments.remove(treatment);
    }
    public String getDescription() {
        StringBuilder description = new StringBuilder("Composite Treatment: ");
        for (Treatment treatment : treatments) {
            description.append("\n - ").append(treatment.getDescription());
        }
        return description.toString();
    }
    public double cost() {
        double totalCost = 0;
        for (Treatment treatment : treatments) {
            totalCost += treatment.cost();
        }
        return totalCost;
    }
}
```

Patrones estructurales – Composite - ejemplo

```
}  
  
// Clase cliente  
public class Client {  
    public static void main(String[] args) {  
        Treatment checkUp = new SingleTreatment("Check-up", 50.0);  
        Treatment xRay = new SingleTreatment("X-Ray", 100.0);  
        Treatment bloodTest = new SingleTreatment("Blood Test", 30.0);  
  
        CompositeTreatment compositeTreatment = new CompositeTreatment();  
        compositeTreatment.addTreatment(checkUp);  
        compositeTreatment.addTreatment(xRay);  
        compositeTreatment.addTreatment(bloodTest);  
  
        System.out.println(compositeTreatment.getDescription() + " costs " +  
compositeTreatment.cost());  
    }  
}
```

¿Cuál es la diferencia entre Decorator y Composite?



Caso práctico



Realizar ejercicio práctico es una excelente manera de aprender y entender cómo elaborar un patrón de diseño estructural. Aquí tienes un ejercicio práctico detallado que puedes seguir:

Ejercicio Práctico:

Caso de estudio: Monedero digital con patrón Decorator

Se desarrolla un monedero digital que permite agregar funcionalidades adicionales a las transacciones básicas. Utilizando el patrón Decorator, se pueden añadir características como "Notificaciones", "Historial de Transacciones" y "Protección contra Fraudes" de manera flexible, sin modificar el código original del monedero. Cada decorador añade una capa adicional de funcionalidad sobre el monedero base, proporcionando una forma modular y extensible de enriquecer las capacidades del sistema.

Objetivos:

1. Elaborar el código java y diagrama de clase UML utilizando el patrón seleccionado.



¿Quién quisiera participar? (2 voluntarios)





¿Qué se les hizo más fácil?
¿Qué se les hizo más retador?





¿Qué hemos aprendido el día de hoy?



Tomar apuntes de manera eficaz ayuda a consolidar el aprendizaje y prepararse para los exámenes.



- **Decorator:** Permite agregar funcionalidades adicionales a un objeto de manera dinámica y flexible, sin alterar su estructura original. Ideal para extender capacidades de un objeto de forma modular.
 - **Composite:** Facilita el manejo de estructuras jerárquicas de objetos, tratando objetos individuales y grupos de objetos de manera uniforme. Ideal para representar y manipular composiciones de objetos en estructuras complejas.
-
-



- Elabora la actividad práctica de acuerdo a la guía de laboratorio de sesión
- Sube la actividad práctica en la plataforma virtual de aprendizaje
- Guarda la actividad con la siguiente etiqueta:
DPA_Actividad09_NombreApellido

Nota: No olvides también revisar tu plataforma UTP+Class



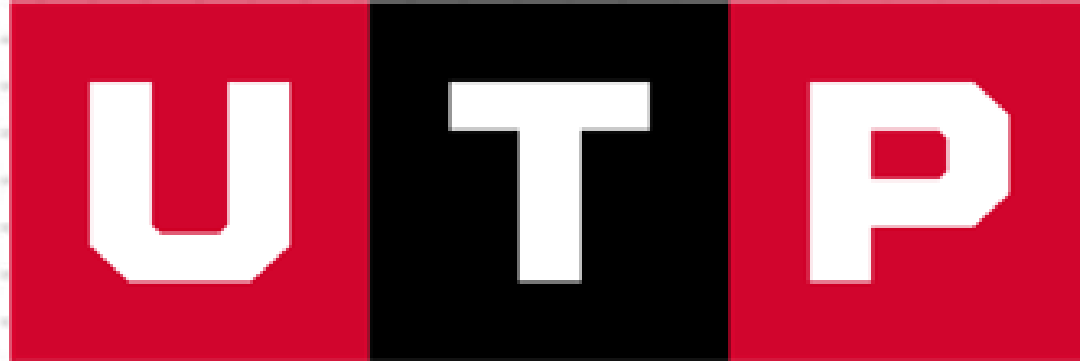
- Hu, C. (2023). *An Introduction to Software Design: Concepts, Principles, Methodologies, and Techniques*. Springer Nature.
- bin Uzayr, S. (2023). *Software Design Patterns: The Ultimate Guide*. CRC Press.

Nota: No olvides también revisar tu plataforma UTP+Class



**MUCHAS GRACIAS QUE
DIOS LOS BENDIGA!!!**





**Universidad
Tecnológica
del Perú**

